LogiBridge Phase 2 Report

FreightBridge Logistics Pvt. Ltd. | LogiEdge Edge AI pilot

Preprocessing experiment, model training, deployment and monitoring

Contents

Section 1	Task C2: Preprocessing pipeline and the shifted-statistics experiment	page 2
Section 2	Task D1: Dataset generation and model training	page 6
Section 3	Task D2: Docker containerisation and OTA demo	page 10
Section 4	Task E1: PSI drift monitoring	page 17
Section 5	Task E2: Ansible deployment playbook	page 22

Each section is a self-contained report with its own calculations, assumptions and open items.

Task C2 — Preprocessing Pipeline

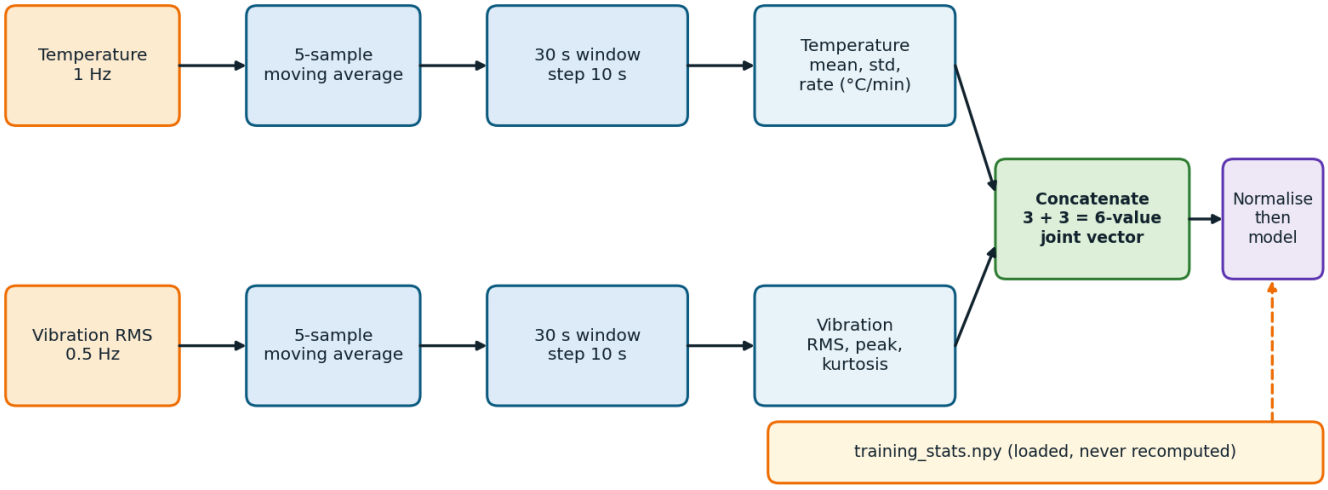
Phase 2 Report section | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component C, Module 3

At a glance

Item	Result
Pipeline	5-sample moving average → 30 s windows, 10 s step → 6 features → z-score with saved statistics
Statistics file	`training_stats.npy`, shape (2, 6), from 10 min of clean Normal-class data (57 windows); loaded at runtime, never recomputed
Mandatory experiment	Correct statistics 99.67%; shifted +3σ 98.38% (−1.29 points); shifted −3σ 91.28% (−8.38 points)
Detection delay	Mean 18.2 s for drift (max 26 s), 6.6 s for vibration; inside the 90 s limit
Verification	32 preprocessing tests pass (plus the 23 simulator tests from Task C1)

1 Pipeline

Each sensor is filtered, windowed and summarised on its own, then fused at feature level



Stage	Specification	Implementation
Filtering	5-sample moving average on temperature and vibration	Causal, one filter per stream. The first four outputs average the samples seen so far, so no sample is lost at start-up.
Windowing	30 s sliding window, step 10 s	Window [s, s+30) is emitted when a sample with timestamp ≥ s+30 arrives: 30 temperature and 15 vibration samples. A window with under 80% of its samples is skipped and counted, never fabricated.
Features	6 values	Three per sensor, extracted independently (Section 2).
Fusion	One joint vector (Task C3)	The two 3-value blocks are concatenated: temperature first, then vibration.
Normalisation	Saved mean and std	z = (x − mean) ÷ std with values loaded from training_stats.npy.

2 The six features and their Normal-class statistics

#	Feature	Definition	Mean	Std
0	temp_mean	Mean of the 30 filtered samples (°C)	4.004	0.055
1	temp_std	Population standard deviation (°C)	0.123	0.026
2	temp_rate_c_per_min	Least-squares slope over the window × 60 (°C/min)	0.002	0.362
3	vib_rms	√(mean of x²) over the filtered samples (g)	0.4454	0.0105
4	vib_peak	max |x| over the filtered samples (g)	0.4717	0.0117
5	vib_kurtosis	Excess kurtosis, m4 ÷ m2² − 3 (a Gaussian gives about 0)	−0.805	0.530

The vibration input is the 0.5 Hz RMS stream from Task C1, so "vibration RMS" here is the RMS of those values over the window. Mean and std are the contents of training_stats.npy.

3 Training statistics: created once, loaded, never recomputed

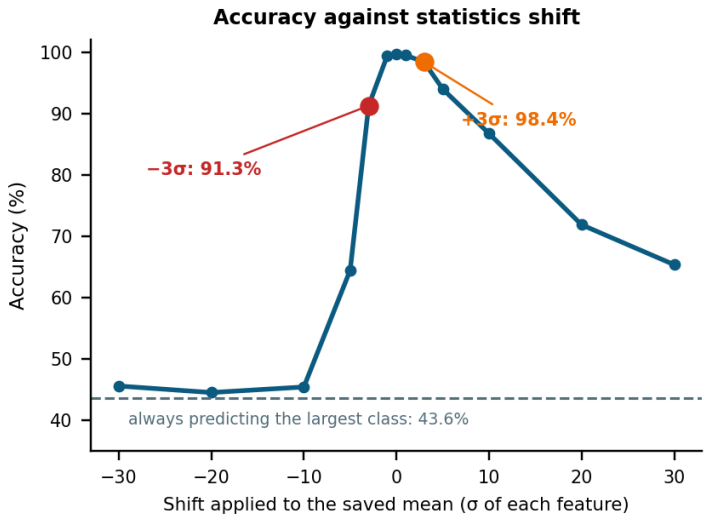
- **Creation (offline):** `python3 preprocessing.py make-stats` generates 10 minutes (600 s) of clean Normal-class data with the Task C1 sensor models, runs it through the pipeline and saves per-feature mean and std (ddof = 1). The 10 minutes yield 57 windows; a 58th would need a sample at t = 600 s.
- **File:** a (2, 6) float64 array; row 0 holds the means, row 1 the standard deviations, in the feature order above.
- **Runtime:** `Normaliser.load()` reads the file. The runtime classes have no fit or update method, the arrays are read-only, and tests check that the runtime source never calls the statistics function or saves a file.
- **Caveat:** windows overlap (30 s windows every 10 s), so the 57 windows are correlated and the standard deviations are uncertain. A longer baseline from real trucks is advisable.

4 Mandatory experiment: inference with correct and shifted statistics

Setup. A small reference classifier (6 → 16 → 3, 163 parameters) stands in for the project model. It was trained five times with different seeds on 4,320 windows from 160 simulated runs and tested on 3,240 windows from 120 separate runs (300 s each, anomaly onset between 20 and 150 s, all four modes). Test classes: Normal 1,413, Warning 744, Critical 1,083. Training and testing both use the saved statistics. "Shifted by 3σ" moves each saved mean by three of its own standard deviations, so every normalised feature moves by 3 (+3σ subtracts 3, −3σ adds 3).

Statistics used	Accuracy (mean ± sd, 5 models)	Change	Normal recall	Warning recall	Critical recall
Correct (saved file)	99.67% ± 0.07	—	99.9%	98.8%	100.0%
Mean shifted +3σ	98.38% ± 0.76	−1.29 points	99.9%	93.7%	99.6%
Mean shifted −3σ	91.28% ± 5.81	−8.38 points	81.5%	98.9%	98.9%
Recomputed from live data (never do this)	43.60%	−56.06 points	—	—	—

Always predicting the largest class would score 43.6%. "Recomputed from live data" normalises each test run with its own windows' mean and std.





Feature shifted by +3σ (others correct)	Physical size of 3σ	Accuracy	Change
temp_std	0.079 °C	98.22%	−1.44 points
temp_mean	0.165 °C	99.29%	−0.38 points
temp_rate_c_per_min	1.09 °C/min	99.32%	−0.35 points
vib_kurtosis	1.59	99.43%	−0.24 points
vib_peak	0.035 g	99.48%	−0.19 points
vib_rms	0.032 g	99.51%	−0.15 points

Findings

- **A 3σ shift costs 1.3 points one way and 8.4 points the other.** The sweep shows accuracy falling steadily with larger shifts, to 86.74% at +10σ and 45.46% at −10σ (no better than always predicting the largest class).
- **The two directions fail differently.** With −3σ, Normal windows look 3σ too high, so 1,308 of 7,065 Normal test windows (18.5%, counted over the five models) were flagged as Warning: false alarms. With +3σ, every window looks 3σ too low, so 169 of 3,720 Warning windows (4.5%) were read as Normal: missed early warnings, the more dangerous error for a cold chain.
- **A small physical shift is enough.** Because the Normal class is tight, 3σ is only 0.17 °C on mean temperature or 0.03 g on vibration RMS. A sensor offset or calibration drift of that size, if not reflected in the saved statistics, moves every input by the same 3σ.
- **Recomputing statistics from live data destroys detection.** Each run is normalised against itself, so the anomaly becomes the new “normal” and accuracy falls to 43.6%. This is why the pipeline only loads the saved file.
- **Spread across models is large at −3σ** (± 5.8 points), so the sensitivity depends on how a particular model places its boundaries. The asymmetry is probably because anomalies only push features upward, so boundaries sit closer to the Normal cluster on the high side. This is an interpretation, not something I tested.
- **Detection stays well inside 90 s.** From fault onset to the first Warning or Critical window, the reference model took on average 18.2 s for temperature drift (max 26 s), 6.6 s for vibration (max 11 s) and 8.7 s for combined (max 12 s), all well inside 90 s, with no run missed.

5 Verification

Check	What it confirms	Result
Moving average	Warm-up and steady-state values; matches numpy convolution	Pass
Features	Known ramp gives 1.2 °C/min; 0.08 °C/s gives 4.8 °C/min; RMS, peak and kurtosis reference values	Pass
Windowing	Ends at 30, 40, 50 s…; step 10 s; 57 windows per 10 min; 30 temperature and 15 vibration samples each	Pass
Filter before features	Window samples equal the moving-averaged stream	Pass
Gaps and bad input	Incomplete windows skipped; duplicate and late messages dropped; unknown stream rejected	Pass
Normaliser	Round trip through .npy; bad shape or zero std rejected; arrays read-only; shift moves z by exactly 3	Pass
Never recomputed	No fit or update method; runtime source never computes or saves statistics; statistics unchanged after live data	Pass
MQTT runtime (paho stub)	Subscribes to temp and vib, publishes features, ignores malformed messages	Pass
Live Mosquitto	Not run: paho-mqtt and Mosquitto are not available in my environment	Please run

6 Assumptions and items to confirm

- **Class labels.** Class 1 is temperature 1–3 °C off setpoint, or a vibration fault (early refrigeration anomaly). Class 2 is more than 3 °C off, or a vibration fault with temperature more than 1 °C off (my reading of "refrigeration unit failure signature"). Please confirm.
- **Meaning of "shifted by 3σ".** I moved the saved mean by 3σ in every feature and report both signs. If you meant shifting the standard deviations too, tell me and I will rerun.
- **Reference classifier.** It is a placeholder. To use your trained model, replace it in `experiments_c2_c3.py` and rerun to refresh these numbers.
- **Data are simulated** with the Task C1 models, so absolute accuracies are optimistic. The statistics should be rebuilt from real Normal-class truck data when available.
- **Definitions.** Standard deviation is the population value; kurtosis is excess kurtosis; the moving average warms up over the first four samples.
- **Door events** are not used, since none of the six specified features comes from them.

Task D1 — Dataset Generation and Model Training

Dataset, MLP training and validation | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component D, Module 4

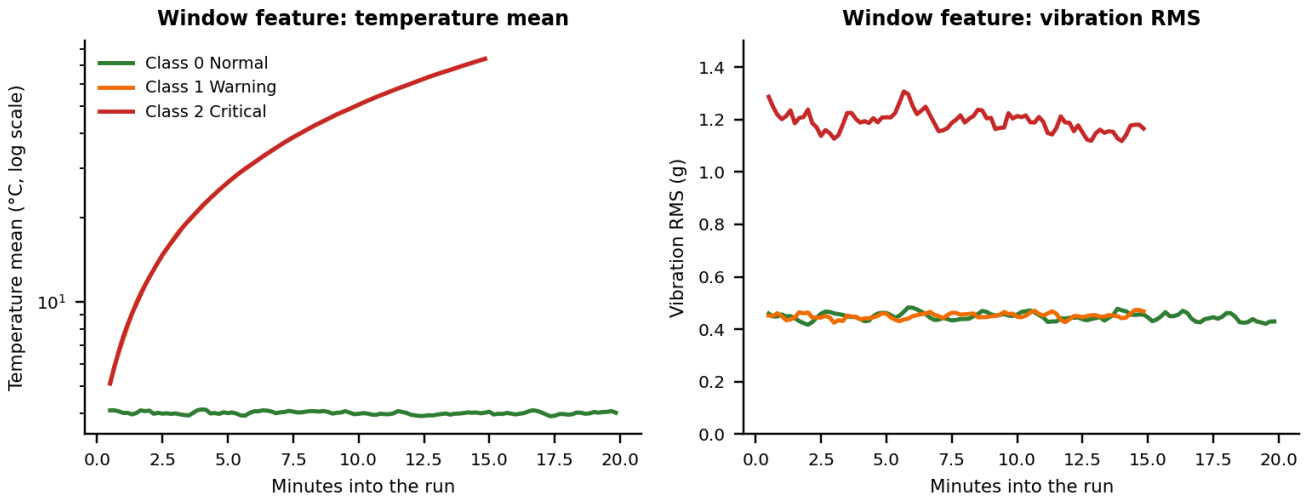
At a glance

Item	Result
Dataset	291 labelled windows: Normal 117, Warning 87, Critical 87 (targets: about 120, 90, 90)
Model	MLP 6 → 32 → 16 → 3, ReLU, softmax output, 803 parameters
Required validation (20% split)	**98.25%** (56/57 windows; 95% interval 90.7% to 99.7%). **Passes the 88% threshold.**
Wider checks	99.2% mean over 50 random splits (lowest 94.7%, none under 88%); 99.3% five-fold; 99.7% on independent runs
Two findings to act on	The model misses some faults that start mid-run, and the table's labels call a 70 °C drift “Warning” (Section 5)

1 Dataset generation

Class	Label	Simulator mode	Duration	Target	Generated	Seed
0	Normal	--anomaly none	20 min	~120	117	101
1	Warning	--anomaly temp_drift	15 min	~90	87	102
2	Critical	--anomaly combined	15 min	~90	87	103

- The simulator loop from Task C1 was run once per mode for the stated duration. Its messages went through the Task C2 pipeline, giving one 6-value feature vector per 30 s window every 10 s, normalised with `training_stats.npy`.
- A 20-minute run gives 117 windows rather than 120 because a window needs its full 30 s (1,200 s → 117; 900 s → 87).
- The run is in-process on an accelerated clock, so it needs no broker. `generate_dataset.py` documents the equivalent live commands for recording from Mosquitto.



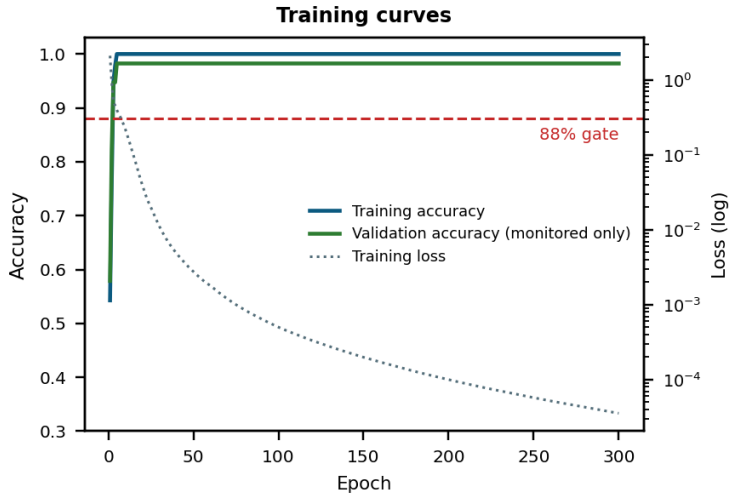
In the left panel the Warning line sits underneath the Critical line, because both runs use the same drift. Vibration RMS is what separates Warning from Critical.

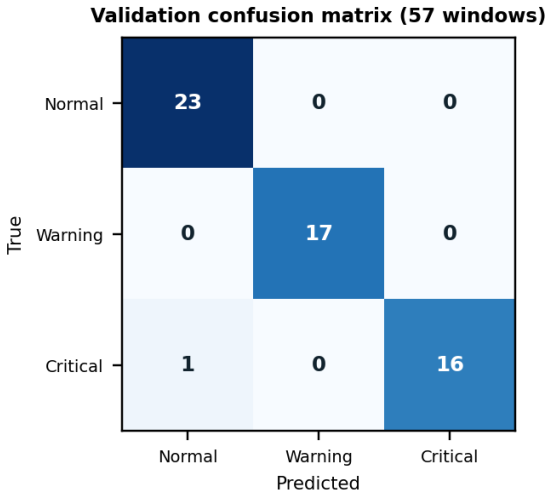
2 Model and training

Setting	Value
Architecture	6 inputs → 32 ReLU → 16 ReLU → 3 softmax outputs (803 parameters)
Input	The 6 normalised features from Task C2, using the saved training_stats.npy
Split	Stratified 80 / 20: 234 training and 57 validation windows (23 Normal, 17 Warning, 17 Critical), seed 42
Training	Adam, learning rate 0.001, batch 32, 300 epochs, cross-entropy loss, He initialisation (first layer scaled by 0.05)
Validation use	Monitored only; it never picks the epoch or any setting
Gate	`train_model.py` exits with an error and saves no model if validation accuracy is 88% or below
Implementation	Pure numpy, no framework needed; weights saved as plain arrays in model_weights.npz; scikit-learn MLP (32, 16) used as a cross-check

3 Results on the required validation set

Class	Windows	Precision	Recall
Normal	23	95.8%	100.0%
Warning	17	100.0%	100.0%
Critical	17	100.0%	94.1%
Overall accuracy	57		**98.25%**



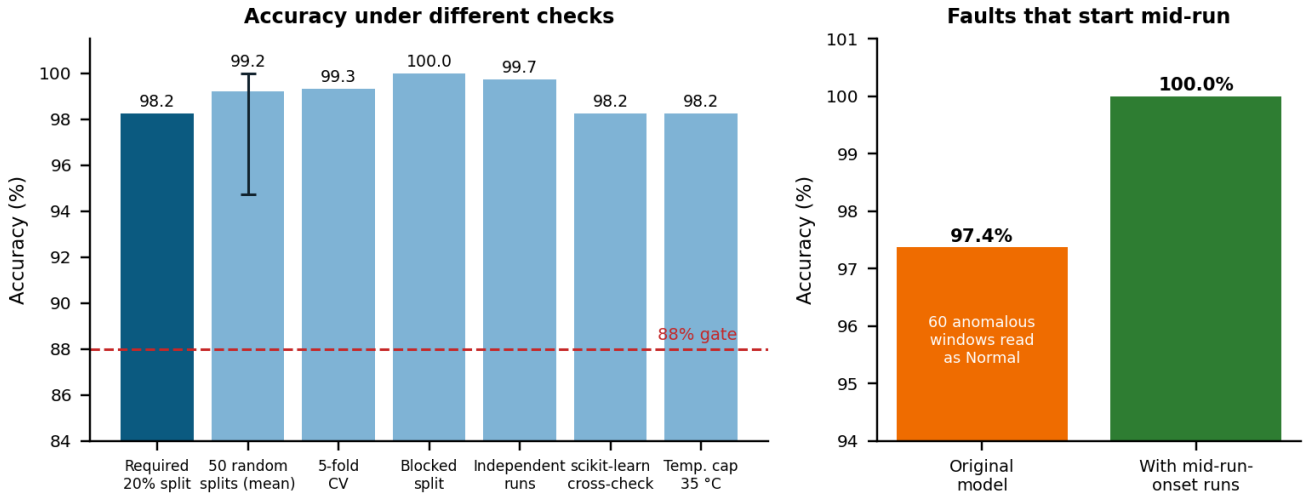


- The one error is a Critical window read as Normal: the very first window of the combined run, where the drift has barely started (mean temperature about 5.1 °C). The model gave Normal with near certainty even though vibration was 80σ above normal.
- With 57 validation windows, each error is 1.75 points, so the interval is wide (90.7% to 99.7%). The next section checks how stable the result is.

4 How far the number can be trusted

Overlapping windows (each shares 20 of 30 s with its neighbour) make a random split optimistic, so the result was checked six more ways.

Check	Accuracy	Note
50 random stratified splits	99.23% ± 1.32	Range 94.7% to 100%; none at or below 88%
Five-fold cross-validation	99.32%	Folds from 98.2% to 100%
Blocked split (last 20% of each run)	100.0%	57/57; a 2-window gap prevents overlap with training
Independent runs (5 new seeds)	99.73%	1,455 windows; lowest set 99.3%; 4 Critical windows read as Normal
scikit-learn MLP (32, 16), same split	98.25%	Same accuracy as the numpy model (56/57)
Temperature capped at 35 °C	98.25%	Physically plausible ceiling instead of unbounded drift



The left axis starts at 84% so differences are visible. The right panel is explained in Section 5.

5 Fitness for cold-chain use: two findings

The model clears the 88% threshold with a wide margin. For a pharmaceutical deployment, two things the single number hides are worth fixing before the model is converted and deployed.

Finding 1: faults that start mid-run can be missed

Test	Original model	Trained with mid-run-onset runs added
Standard runs (fault from t = 0), 5 independent sets	99.73%	100.00%
Runs where the fault starts at 80–320 s, 5 independent sets	97.37%	100.00%
Anomalous windows read as Normal (of 1,520)	60 (3.9%)	0
Required validation split	98.25%	100.0% (57/57)

The training runs start every fault at t = 0, so they contain no windows between “normal” and “fully drifting”, and the model behaves unpredictably there. Adding runs with `--anomaly-start` fixes it. This model is saved as `model_weights_onset.npz` (`train_model.py --augment-onset`) and also passes the gate. **I recommend carrying it forward.**

Finding 2: the table's labels disagree with the Task C class definitions

The assignment labels by simulator mode, and the drift is unbounded, reaching about 76 °C. By the Task C thresholds (Warning is 1–3 °C off setpoint, Critical is more than 3 °C), 86 of the 87 “Warning” windows are already Critical. The model learns what the table says, so a pure temperature breach reads as Warning.

Truth by Task C thresholds (independent runs)	Windows	Predicted Normal	Predicted Warning	Predicted Critical
Normal	351	351	0	0
Warning	3	0	3	0
Critical	519	3	258	258

Agreement with threshold-based labels is 70.1%: 258 of 519 temperature-breach windows are called Warning where Task C says “immediate escalation required”. Options: (1) add a rule after the model that forces Critical when mean temperature exceeds setpoint + 3 °C, (2) relabel by thresholds, or (3) generate shorter drift episodes so Warning contains 1–3 °C windows. I recommend (1) with (3). Please choose.

6 Files and how to run

File	Purpose
generate_dataset.py	Runs the simulator in the three modes and writes training_dataset.npz and .csv
train_model.py	Trains the 6-32-16-3 MLP, applies the 88% gate, saves model_weights.npz and model_metadata.json
model_weights.npz	The model required by the task (original data)
model_weights_onset.npz	Recommended model (mid-run-onset runs added); metadata in model_metadata_onset.json
validation_checks.py	The wider checks in Section 4 and the onset and label tests in Section 5
test_training.py	21 tests; the full project suite is 76 tests, all passing

```
python3 preprocessing.py make-stats          # training_stats.npy (Task C2), once
python3 generate_dataset.py                  # 291 labelled windows
python3 train_model.py                       # exits with an error below 88%
python3 train_model.py --augment-onset --out model_weights_onset.npz --meta model_metadata_onset.json
python3 validation_checks.py                 # wider checks (about 1 minute)
python3 -m unittest -v test_training
```

7 Assumptions and items to confirm

- **Labelling policy.** Which option do you want for Finding 2? The required model follows the table as written.
- **Which model to carry forward** to conversion and deployment: the required one or the onset-trained one (recommended).
- **Simulated data.** Near-perfect accuracy mostly reflects how separable the simulator's modes are. It is not evidence of performance on real trucks, so real data should be added when available.
- **Framework.** The model is plain numpy. For conversion I can load the saved weights into an equivalent Keras or PyTorch model and check the outputs match; tell me which toolchain you will use.
- **Seeds and temperature.** Seeds 101–103 are fixed for reproducibility. Temperature drift is unbounded as in Task C1; use `--temp-cap 35` for a plausible ceiling.

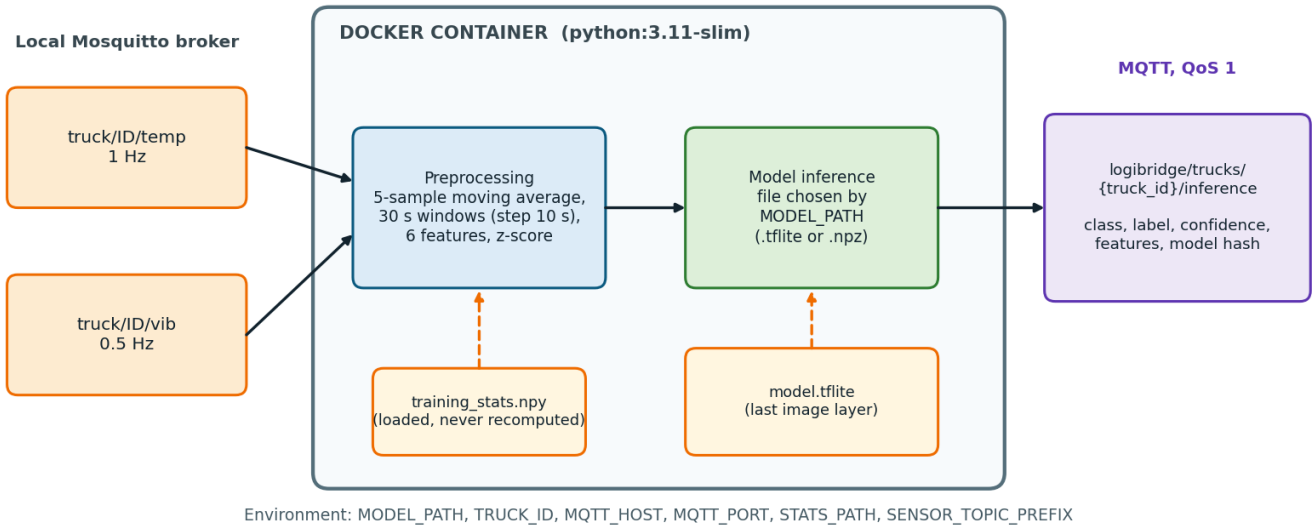
Task D2 — Docker Containerisation and OTA Demo

Inference container and layer-cached model updates | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component D, Module 4

At a glance

Item	Result
Container	`python:3.11-slim`; preprocessing → model inference → MQTT publish to `logibridge/trucks/{truck_id}/inference`
Layer order	Every pip install layer comes before `COPY model.tflite .`, which is the last layer that carries data
Model switching	`MODEL_PATH` selects the model at run time; no rebuild
OTA layer cache	A model-only change rebuilds exactly one data layer (predicted by applying Docker's cache rule to the real Dockerfile)
Bandwidth, 85 trucks	About 5,523.6 MB saved of 5,525 MB (99.98%), Rs 552.36 at Rs 0.10/MB. Image sizes are my estimates; replace with measured values.
Status	**Not built or run on Docker, and no model.tflite yet:** Docker and TensorFlow were not available to me. All 174 project tests pass for the logic (Section 7).

1 What the container does



Variable	Default	Purpose
MODEL_PATH	/app/model.tflite	Model file (.tflite, or .npz weights). Change it to switch variants without rebuilding
TRUCK_ID	T001	Used in the output topic logibridge/trucks/{truck_id}/inference
MQTT_HOST, MQTT_PORT	localhost, 1883	Local Mosquitto broker (in compose: mosquitto)
STATS_PATH	/app/training_stats.npy	Saved normalisation statistics (loaded, never recomputed)
SENSOR_TOPIC_PREFIX	truck/{truck_id}	Where the sensors publish; the service reads <prefix>/temp and <prefix>/vib as in Task C1
REFERENCE_DIST_PATH, DRIFT_SCORE	unset, normal_confidence	PSI drift monitoring (Task E1); off unless a reference file is given

```
logibridge/trucks/T001/inference  (QoS 1)   [example message]
{"truck_id":"T001","window_end":"2027-01-15T08:00:30.000Z","class":2,"label":"Critical","confidence":0.9998,
 "probabilities":[0.0,0.0002,0.9998],"features":{"temp_mean":5.1,...,"vib_kurtosis":0.8},
 "model":{"file":"model.tflite","sha256":"3f9c1a2b7d40"}}
```

2 Dockerfile and layer order

```
FROM python:3.11-slim
ENV PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1 \
    MODEL_PATH=/app/model.tflite \
    STATS_PATH=/app/training_stats.npy \
    TRUCK_ID=T001 \
    MQTT_HOST=localhost \
    MQTT_PORT=1883
RUN useradd --system --uid 10001 --no-create-home logiedge
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY preprocessing.py drift_monitor.py inference_service.py training_stats.npy ./
COPY model.tflite .
USER logiedge
CMD ["python", "inference_service.py"]
```

Step	Holds	Changes when	Model-only change
FROM, ENV, useradd, WORKDIR	Base image, settings, non-root user	Rarely	Cached
COPY requirements.txt	Dependency list	A package is added or updated	Cached
RUN pip install	numpy, paho-mqtt, tflite-runtime	requirements.txt changes	Cached
COPY code + statistics	preprocessing.py, inference_service.py, training_stats.npy	Code or statistics change	Cached
COPY model.tflite .	The model	Every model update	**Rebuilt: the only data layer**
USER, CMD	Metadata	Never	Re-run, 0 bytes

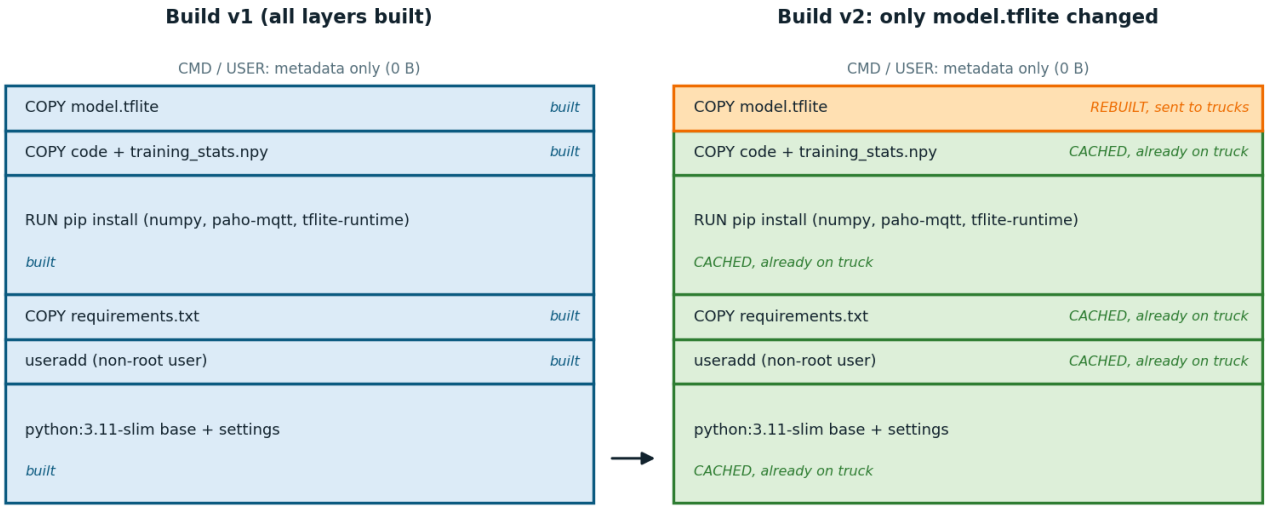
3 Switching model variants without a rebuild

```
# default model baked into the image
docker run --rm --network host logiedge/inference:v1

# another variant from a mounted folder, same image, no rebuild
docker run --rm --network host -v "$PWD/models:/models:ro" \
           -e MODEL_PATH=/models/model_onset.tflite logiedge/inference:v1
```

The model hash in each result message (`model.sha256`) shows which variant produced it, which makes the switch visible on screen. A wrong path stops the container with a clear error (exit code 2) instead of running with no model.

4 OTA layer-cache demonstration



Run it (needs Docker): `./ota_demo.sh` builds v1 with no cache, replaces only model.tflite, rebuilds v2, compares the image layer digests and prints the bandwidth figures. In the real build log, steps 1–7 show `CACHED` and only `COPY model.tflite .` is executed. The digest comparison should report all layers identical except one.

Prediction from the Dockerfile (Docker's cache rule applied by `layer_cache_check.py` to two build contexts that differ only in model.tflite):

Step	Instruction	Result
1–4	FROM, ENV, RUN useradd, WORKDIR	CACHED
5	COPY requirements.txt .	CACHED
6	RUN pip install -r requirements.txt	CACHED
7	COPY preprocessing.py inference_service.py training_stats.npy ./	CACHED
8	COPY model.tflite .	REBUILT (only data layer)
9–10	USER logiedge, CMD [...]	REBUILT (metadata, 0 bytes)

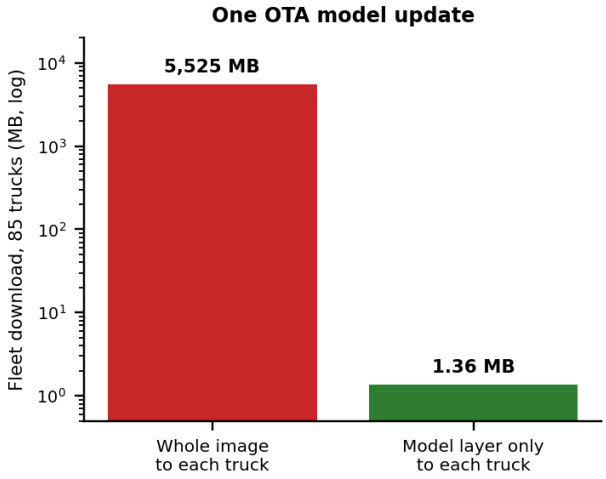
Condition for the saving: *trucks only skip the cached layers if the rebuilt image reuses the same layers, so build on the same builder or use a shared registry cache (`--cache-from`). A clean build on a new machine produces new layer digests and trucks would download everything.*

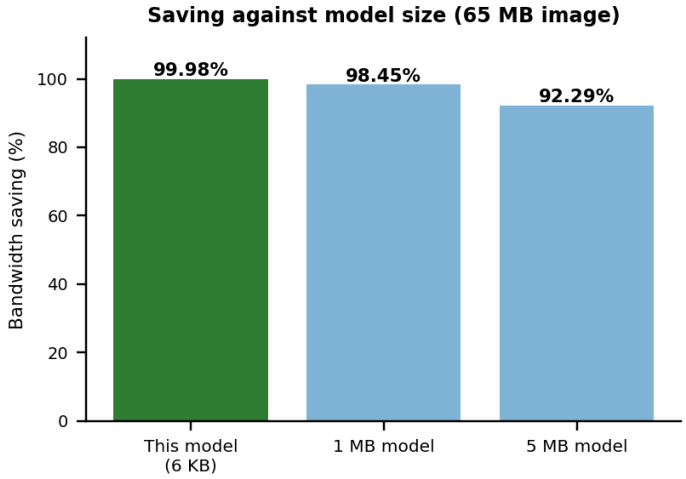
5 Bandwidth saving for 85 trucks

A truck that already runs the previous image downloads only the layers it lacks. With the model as the last layer, that is the new model layer plus a small manifest and config. Sizes below are my estimates, not measurements: `ota_demo.sh` measures the real ones and `ota_bandwidth.py` recomputes everything.

Assumption	Value	Basis
Full image, compressed	65 MB (range 62–67)	python:3.11-slim about 45–46 MB (copies on Docker Hub: 45.3 MB arm64, 45.7 MB amd64) + numpy wheel 14.2 MB aarch64 or 18.3 MB x86_64 (PyPI) + tflite-runtime about 2–3 MB (size not found; assumed)
Model layer, compressed	6 KB	803 float32 parameters (3.2 KB) plus file structure (estimate)
Manifest and config per update	10 KB	Estimate
Tariff, units	Rs 0.10 per MB; 1 MB = 1,000,000 bytes	As in Task A1

Step	Calculation	Result
Full update per truck	Whole image	65.000 MB
Full update, 85 trucks	85 × 65 MB	5,525.00 MB; Rs 552.50
Layer update per truck	6 KB + 10 KB = 16 KB	0.016 MB
Layer update, 85 trucks	85 × 0.016 MB	1.36 MB; Rs 0.14
Saving, 85 trucks	5,525.00 − 1.36	**5,523.64 MB (99.98%); Rs 552.36**





The saving shrinks as the model grows (a 1 MB model saves 98.45%, a 5 MB model 92.29%) but stays large. Across the 62–67 MB range the saving is 5,269 to 5,694 MB (99.97% to 99.98%). For scale, one full 65 MB image pull equals about 1,916 days of a truck's alert traffic from Task A1 (0.0339 MB per day). At 265 trucks the same update saves 17,221 MB.

6 Demo video script (about 3 minutes)

I cannot record video, so this is a ready-to-film script. Replace the numbers with those printed by `ota_demo.sh` if they differ.

Time	On screen	Say
0:00	Dockerfile	The image starts from python:3.11-slim. All pip installs come first, the model is copied last, and MODEL_PATH lets us pick a model at run time.
0:25	docker compose up; mosquitto_sub on logibridge/trucks/#; simulator running	The container preprocesses the sensor stream, runs the model and publishes each result to logibridge/trucks/T001/inference.
1:00	Run with -e MODEL_PATH=/models/model_onset.tflite	Same image, different model, no rebuild: the model hash in the messages changes.
1:25	./ota_demo.sh: build v1, replace model.tflite, build v2	Now an over-the-air update. I change only the model file and rebuild.
1:55	Build log with CACHED lines; layer-digest comparison	Every step is cached except COPY model.tflite. All layers are identical except one, so a truck downloads only that layer.
2:25	ota_bandwidth.py output	For 85 trucks, sending the whole image would be about 5,525 MB. Sending only the model layer is about 1.36 MB. That saves 5,524 MB, 99.98%, about Rs 552 at Rs 0.10 per MB.

7 Verification: what ran and what did not

Check	How	Result
Dockerfile rules: base image, pip before model, model last data layer, MODEL_PATH, non-root user, copy sources, .dockerignore, pinned requirements	Static tests	Pass
Docker cache rule on the real Dockerfile: model-only change rebuilds one data layer; code change also rebuilds the model; requirements change rebuilds pip	Cache analysis on two build contexts	Pass
Service: topic, payload, QoS 1, subscriptions, last will and status, malformed input, failing model, clean stop	paho stub	Pass
MODEL_PATH switching: two variants give different model hashes; startup errors return code 2	Tests	Pass
numpy backend gives the same classes as the training code; results follow the simulator mode	Tests	Pass
TFLite path: float32 input and output; int8 input quantised and output dequantised	Fake interpreter	Pass
Bandwidth arithmetic	Tests	Pass
Package pins exist with Python 3.11 wheels for x86_64 and aarch64: numpy 1.26.4, paho-mqtt 2.1.0 (pure Python), tflite-runtime 2.14.0; tflite-runtime needs numpy 1.23.2 or newer and glibc 2.34+ on aarch64, which python:3.11-slim (Debian 13) has	Checked against PyPI and Docker Hub listings; not installed	Checked
TensorFlow 2.14 supports Python 3.9–3.11 and needs numpy 1.23.5 or newer; the "FULLY_CONNECTED version 12" mismatch error between a new converter and an old runtime is a documented failure	Checked in project files and issue reports	Checked
paho 2.x callback signatures (on_connect, on_disconnect) match the code; eclipse-mosquitto:2 needs a mounted config with a listener and allow_anonymous for the demo; BuildKit plain output prints "CACHED"	Checked in documentation	Checked
ai-edge-litert fallback: cp311 wheels exist for x86_64 and aarch64 (glibc 2.17+)	Checked against PyPI	Checked
docker build, real layer cache, image size	Needs Docker	Not run
convert_to_tflite.py and the real TFLite runtime	Needs TensorFlow	Not run
Live Mosquitto and an actual pip install	Needs network and broker	Not run

8 Items to confirm

- **Create model.tflite first:** in a Python 3.11 environment run `pip install tensorflow==2.14.0 numpy==1.26.4 tflite-runtime==2.14.0` then `python3 convert_to_tflite.py`. It refuses to write the file unless the TFLite model matches the numpy model on every dataset window. Tell me the result and I will fix anything that fails.
- **Topic names.** The output topic follows your specification (logibridge/trucks/...), while the sensors still publish on truck/ID/... from Task C1. Set SENSOR_TOPIC_PREFIX if you move them. Please confirm the "logibridge" spelling is intended.
- **Package pins (checked online).** numpy 1.26.4 has Python 3.11 wheels for x86_64 and aarch64. paho-mqtt 2.1.0 is pure Python and the latest release. tflite-runtime 2.14.0 has Python 3.11 wheels for x86_64, aarch64 and armv7l and needs numpy 1.23.2 or newer. numpy stays below 2.0 because that tflite-runtime (October 2023) predates NumPy 2.0; this last point is my inference, not a reported failure. TensorFlow 2.14 needs Python 3.9–3.11 and numpy 1.23.5 or newer with no upper bound, so keep the numpy pin when installing it. I did not run pip install.
- **Two risks from that check.** (1) tflite-runtime 2.14.0 is the last release, so convert with TensorFlow 2.14 (it has Python 3.11 wheels but none for 3.12), because a model from a much newer TensorFlow can fail to load on the older runtime; convert_to_tflite.py now also tests with tflite-runtime if installed. (2) The aarch64 wheel needs glibc 2.34 or newer, which python:3.11-slim provides (it is currently Debian 13); do not switch to an Alpine or Debian 11 base. If tflite-runtime gives you trouble, ai-edge-litert has Python 3.11 wheels for x86_64 and aarch64, at the cost of a larger image.
- **Sizes.** The base-image and numpy sizes now come from published listings (about 62–67 MB in total). The tflite-runtime wheel, the 6 KB model layer and the 10 KB manifest are still estimates, so run ota_demo.sh to replace them with measured values before filming.
- **INT8 later.** Normalised inputs run from about −3 to 1,300, so full-integer quantisation can crush the Normal range; validate accuracy before shipping a quantised model.

Task E1 — PSI Drift Monitoring

Population Stability Index on model output scores | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component E, Module 5

At a glance

Item	Result
Monitor	PSI over 4 score bins, on the last 100 inferences, every 60 s; prints the PSI each time and `[LOGIBRIDGE DRIFT ALERT] PSI=1.234` above 0.25
Reference	`reference_dist.json` from 300 clean Normal-class windows
Literal "confidence" (max softmax)	**Does not work for this model:** it is 0.999 or more on clean and on faulty windows, so PSI stays 0.000 and never reaches 0.25
Recommended score: Normal-class confidence	Crosses 0.25 **1 min 30 s** after injection (limit 5 min); back below 0.10 **16 min 30 s** after restore; no false alarms; identical over 10 seeds
Status	Ran end to end in-process (no broker). The container path was tested with a paho stub. I cannot record the video; a script is in Section 6.

1 What was built

Requirement	Implementation
Reference from 300 clean Normal windows, 4 bins, saved as reference_dist.json	`drift_monitor.py make-reference` runs the model on 300 windows from the simulator in `--anomaly none` mode (seed 5001); bins [0, 0.25), [0.25, 0.50), [0.50, 0.75), [0.75, 1.0]
PSI on a rolling window of the last 100 inferences	`PsiMonitor` keeps the last 100 scores; PSI = Σ (current − reference) × ln(current ÷ reference), zero proportions floored at 0.0001
Every 60 seconds; print the current PSI	Evaluated every 60 s of data time (from message timestamps, so it also works when the simulator is sped up). Prints the PSI, or "collecting n/100" until 100 inferences exist
Alert when PSI > 0.25	Prints exactly `[LOGIBRIDGE DRIFT ALERT] PSI=1.234` on every evaluation above 0.25
Inject --anomaly combined mid-run, then restore clean data	`drift_demo.py` uses the new `--anomaly-start` and `--anomaly-end` simulator options; the monitor also runs inside the inference container and publishes each evaluation on logibridge/trucks/{truck_id}/drift

2 Reference distribution

Score	[0, 0.25)	[0.25, 0.50)	[0.50, 0.75)	[0.75, 1.0]	Mean	Lowest
Max-softmax confidence	0	0	0	300	0.9999	0.9987
Normal-class confidence	0	0	0	300	0.9999	0.9987

All 300 windows fall in the top bin for both scores (model model_weights.npz, hash 3013acfc9863). The two scores are equal on clean data because a Normal window's most likely class is Normal.

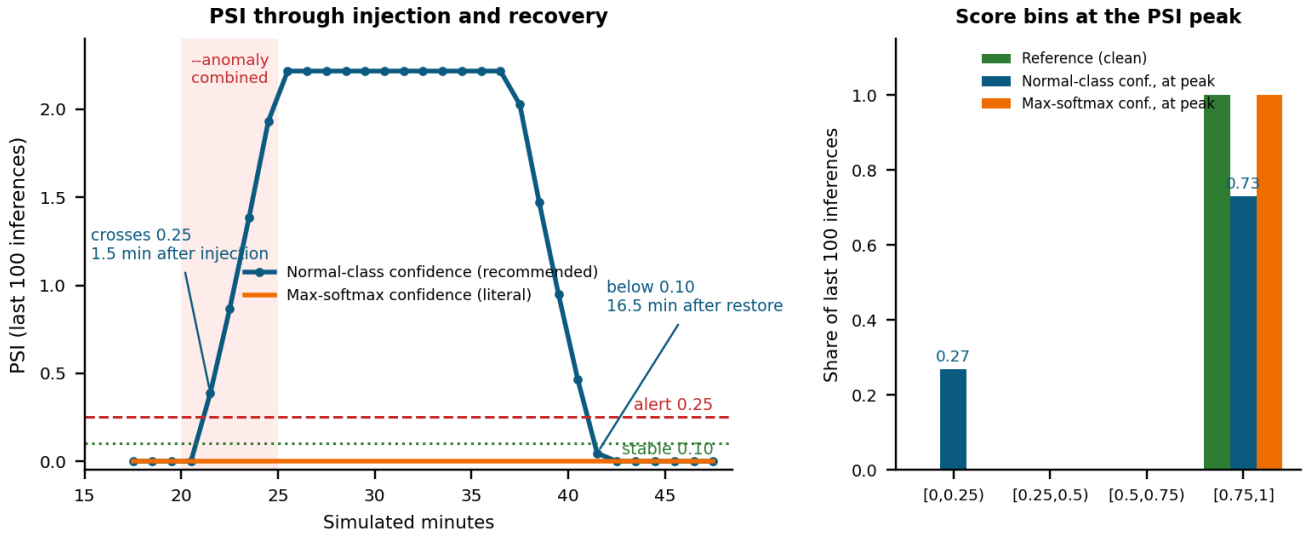
3 Why the literal confidence score cannot detect this drift

Test windows (fixed seeds)	Mean confidence	Lowest	Bin shares [0, 0.25) … [0.75, 1.0]
Clean (--anomaly none)	0.9999	0.9994	0, 0, 0, 100%
Faulty (--anomaly combined)	0.9999	0.9906	0, 0, 0, 100%
Clean, scaled (T=2.5)	0.9741	0.9315	0, 0, 0, 100%
Faulty, scaled (T=2.5)	0.9929	0.8560	0, 0, 0, 100%

- A confident classifier is as sure about “Critical” as it is about “Normal”, so a clean separable fault leaves the max-softmax distribution unchanged. Max-softmax confidence detects uncertain inputs, not a shift to another confident class.
- Calibrating does not help. The temperature that minimises the loss on held-out data is 2.5 (747 windows, 13 errors; `confidence_check.py`); it lowers the confidences a little but every window stays in the top bin, so PSI is still about 0.
- The result is therefore a property of the model and the specified bins, not of the monitor. The monitor code is correct (the tests feed it shifted scores and it fires).

4 Recommended score: Normal-class confidence

The monitored score is the model's probability that the cargo is Normal (a confidence in [0, 1] that the load is fine). It is near 1 on clean data and near 0 when a fault appears, so the output distribution visibly moves. This is the closest change to the specification that works; please confirm it is acceptable (Section 8).



Left: PSI against simulated time. Right: the score bins for the last 100 inferences when the window holds 27 non-Normal windows.

Event	Simulated time	PSI
Window full, clean data	17 min	0.000 (no false alarms before injection)
--anomaly combined injected	20:00	0.000 at 20:30
First alert	21:30 (1 min 30 s after injection)	0.387
Anomaly ends, clean data restored	25:00	2.217 (peak)
PSI starts falling as faulty windows roll out	37:30	2.027
Back below 0.10	41:30 (16 min 30 s after restore)	0.046, then 0.000

Console output from drift_demo.py (excerpt):

```
[LOGIBRIDGE] t=00:19:30 PSI=0.000 status=stable (n=100)
=== t=00:20:00  INJECT: --anomaly combined activated ===
[LOGIBRIDGE] t=00:20:30 PSI=0.000 status=stable (n=100)
[LOGIBRIDGE] t=00:21:30 PSI=0.387 status=drift (n=100)
[LOGIBRIDGE DRIFT ALERT] PSI=0.387
[LOGIBRIDGE] t=00:22:30 PSI=0.865 status=drift (n=100)
[LOGIBRIDGE DRIFT ALERT] PSI=0.865
=== t=00:25:00  RESTORE: clean data back ===
[LOGIBRIDGE] t=00:36:30 PSI=2.217 status=drift (n=100)
[LOGIBRIDGE] t=00:37:30 PSI=2.027 status=drift (n=100)
[LOGIBRIDGE] t=00:40:30 PSI=0.463 status=drift (n=100)
[LOGIBRIDGE] t=00:41:30 PSI=0.046 status=stable (n=100)
[LOGIBRIDGE] t=00:42:30 PSI=0.000 status=stable (n=100)
```

5 How to read it, and its limits

Non-Normal windows in the last 100	1	2	3	4	5	6
PSI (floor 0.0001)	0.046	0.106	0.171	0.241	0.313	0.387
Status	stable	moderate	moderate	moderate	**alert**	alert

- **Alert when 5% of recent windows leave the Normal bin.** Because the reference sits entirely in one bin, PSI behaves like a count of non-Normal windows; five in the last 100 (about 8 minutes of data) cross 0.25.
- **Recovery takes about 17 minutes by design.** A window of 100 inferences at one inference per 10 s holds 1,000 s of history, so faulty windows only roll out 16 min 30 s after the restore.
- **The floor matters.** With a floor of 0.01 the alert comes at 2 min 30 s and the peak PSI is 0.94; with 0.001 it is 2 min 30 s; with 0.0001 or 0.000001 it is 1 min 30 s. Recovery is 16 min 30 s in every case.
- **It cannot tell a real fault from a model problem.** A genuine Critical event also moves the output distribution. Treat PSI as “the mix of predictions has changed” and read it alongside the class alerts. Adding PSI on the six input features would also catch drift the output hides, such as a sensor offset that stays within what the model calls Normal.

6 Running it and the demo video

```
# A. In-process demo, no broker (add --pace 0.25 when filming)
python3 drift_demo.py
python3 drift_demo.py --score max_confidence        # the literal score: PSI stays 0.000

# B. Live: broker, inference container with monitoring on, and the simulator (three terminals)
docker compose up                                     # see Task E2 for the Ansible-managed version
docker logs -f logiedge-inference-1                   # PSI lines appear here
python3 simulator.py --anomaly combined --anomaly-start 1200 --anomaly-end 1500 \
                          --speed 30 --duration 2900  # the same timeline in about 100 s
```

Time	On screen	Say
0:00	reference_dist.json and the PSI definition	The reference is the score distribution on 300 clean Normal windows. PSI over the last 100 inferences is printed every 60 seconds.
0:25	Demo running on clean data: PSI=0.000 lines	On clean data PSI is zero.
0:50	INJECT line, then the first DRIFT ALERT	I activate the combined anomaly. PSI crosses 0.25 after 1 min 30 s, well inside the 5-minute limit.
1:15	RESTORE line, PSI falling	Clean data is restored. The 100-inference window flushes, so PSI falls and is back below 0.10 after about 16 and a half minutes.
1:40	Run with --score max_confidence	The literal confidence score never moves for this model, which is why I monitor the Normal-class confidence.

7 Verification

Check	How	Result
PSI maths, bin edges (1.0 in the last bin), zero floor, status thresholds	Known values	Pass
Reference file: 300 windows, bins, model hash matches model_weights.npz, rejects bad files	Tests	Pass
Monitor: every 60 s of data time, last 100 inferences, warm-up, exact alert text only above 0.25	Tests	Pass
Demo: crosses 0.25 within 5 min, no false alarm, back below 0.10, output lines present	End-to-end run	Pass
Literal max-softmax score stays below 0.01 PSI and never alerts	End-to-end run	Pass (shows the limitation)
Inference service prints PSI lines, publishes on the drift topic, survives a bad reference file	paho stub	Pass
Simulator --anomaly-end restores clean data; drift restarts from zero	Tests	Pass
Live Mosquitto and container run; the video	Needs Docker and a recorder	Not run

8 Items to confirm

- **Which score to submit.** The literal max-softmax confidence cannot meet the “PSI must cross 0.25” requirement with this model. I recommend the Normal-class confidence; the code supports both (`DRIFT_SCORE`), and the report shows both results.
- **Model.** The reference and demo use model_weights.npz. If you carry the onset-trained model forward, regenerate the reference with `drift_monitor.py make-reference --model model_weights_onset.npz`; the reference stores the model hash so a mismatch is visible.
- **Floor and warm-up.** I used a floor of 0.0001 and print “collecting n/100” until 100 inferences exist (about 17 minutes). Tell me if you would rather evaluate on a partial window.
- **Data time.** The 60 s schedule follows message timestamps, which equals wall-clock time on a real truck.

Task E2 — Ansible Deployment Playbook

logibridge_deploy.yml: 7 tasks, run twice | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component E, Module 5

At a glance

Item	Result
Playbook	`logibridge_deploy.yml`: one play, exactly 7 tasks in the required order
Second run	Designed to report **changed=0**: the stop task runs only when a file changed, the pull uses a version tag, and the start task is declarative
Verified here	Structure, Jinja templates, and the two shell tasks run against a fake docker command (21 tests pass)
Not verified	**The playbook has not been run by Ansible**, and no changed=0 output exists yet: Ansible and Docker were not available to me. Run `./run_playbook_twice.sh` and record it (Section 5).

1 The seven tasks

#	Task	Module	Reports “changed” when	Second run
1	Create /opt/logibridge	file	the directory is missing	ok
2	Copy model to /opt/logibridge/model.tflite	copy	the file content differs (checksum)	ok
3	Copy reference_dist.json	copy	the file content differs (checksum)	ok
4	Stop the container if running	shell	a running container was stopped; runs only if task 2 or 3 changed	skipped
5	Pull the image from the local registry	docker_image	the tag was not on the truck yet	ok
6	Start the container with environment variables	docker_container	it was stopped, missing, or its image or settings differ	ok
7	Wait 15 seconds and verify it is running	shell	never (fails if the container is not running)	ok

The container receives MODEL_PATH, REFERENCE_DIST_PATH, DRIFT_SCORE, TRUCK_ID, MQTT_HOST and MQTT_PORT, and mounts /opt/logibridge read-only, so a new model file takes effect with no image rebuild (the Task D2 MODEL_PATH feature) and the Task E1 monitor reads the same reference file.

2 Why a straightforward playbook fails the second-run test

The obvious version stops the container (task 4), pulls (task 5) and starts it (task 6) on every run. A stop task that always stops, followed by a start that always starts, reports two changes on a run where nothing changed. Three decisions avoid that:

Decision	Reason
Task 4 is a shell command that stops only a container that is running, and runs only when the model or reference file changed	Ansible's docker_container with state=stopped would create a missing container first (documented behaviour), which is wrong on a first deployment. The shell version reports changed only if it stopped something.
Task 5 pulls a version tag (v1, v2) instead of latest	A tag already on the truck is not pulled again, so repeat runs change nothing. A new image is rolled out by changing image_tag. Set force_pull=true only for a mutable tag.
Task 6 declares the end state (started, with this image and environment)	Ansible then does nothing if it already matches, starts it if task 4 stopped it, and recreates it if the image differs, so no separate restart is needed.

3 What to expect

	First deployment	Second run (no changes)	New model file	New reference file only	New image tag
1 Create /opt/logibridge	**changed**	ok	ok	ok	ok
2 Copy model.tflite	**changed**	ok	**changed**	ok	ok
3 Copy reference_dist.json	**changed**	ok	ok	**changed**	ok
4 Stop container if running	ok	skipped	**changed**	**changed**	skipped
5 Pull image from registry	**changed**	ok	ok	ok	**changed**
6 Start container with env	**changed**	ok	**changed**	**changed**	**changed**
7 Wait 15 s, verify running	ok	ok	ok	ok	ok

Predicted from the playbook's conditions, not observed. Second-run column outlined: changed=0 on every task.

Run	Expected PLAY RECAP line (not observed)
First	ok=7 changed=5 unreachable=0 failed=0 skipped=0
Second, no changes	**ok=6 changed=0 unreachable=0 failed=0 skipped=1**

The first run is expected to change tasks 1, 2, 3, 5 and 6 on a clean machine. The second run takes at least 15 seconds because of task 7. If your recap differs, send it to me.

4 Running it

```
# once: tools, local registry, model and reference
pip install ansible docker && ansible-galaxy collection install community.docker
python3 convert_to_tflite.py                        # model.tflite (Task D2)
docker build -t logiedge/inference:v1 . && ./prepare_demo_registry.sh   # localhost:5000/logiedge/inference:v1

# the demo: both runs, then the changed=0 check
./run_playbook_twice.sh              # add -K if sudo asks for a password, or -e use_become=false

# a real update later
cp new_model.tflite model.tflite && ansible-playbook -i inventory.ini logibridge_deploy.yml
```

5 Demo video script

Time	On screen	Say
0:00	logibridge_deploy.yml, scrolling the 7 task names	The playbook has exactly seven tasks, from creating the directory to verifying the container.
0:30	`./run_playbook_twice.sh`, run 1 recap	The first run deploys: the directory, model, reference and image are new, so five tasks change.
1:10	Run 2 starts; the 15-second wait	Now I run it again with nothing changed in between.
1:40	Run 2 recap and the PASS line	The second run reports changed=0. The stop task is skipped because no file changed, and nothing else needs to change.
2:00	docker ps; docker logs showing PSI lines	The container is running with the new model and the drift monitor from Task E1.

6 Verification

Check	How	Result
One play, exactly 7 tasks, required names and order; no handlers, roles or includes	Tests	Pass
Correct modules; paths /opt/logibridge, model.tflite, reference_dist.json; pull from localhost:5000; environment variables	Tests	Pass
The container's configuration reader accepts the environment the playbook sets	Tests	Pass
Jinja templates render, including Docker's {{.State.Running}} inside a raw block	Rendered with jinja2	Pass
Stop guard: skipped when no file changed; changed only when a container was actually stopped	Condition expressions evaluated	Pass
Stop and verify shell tasks: stop a running container, do nothing when stopped or missing; verify fails unless running	Real scripts, fake docker	Pass
Module behaviour (docker_container stopped, image-change recreate, docker_image pull) matches the documentation	Read in the collection docs	Checked
Ansible run, real Docker, changed=0 on the second run, the video	Needs Ansible and Docker	Not run

7 Items to confirm

- **Image tag.** Pulling a version tag is what keeps the second run at changed=0. If you want “latest”, use `-e force_pull=true`; I have not tested whether your collection version then reports changed=false for an unchanged image.
- **Task 4 runs only when a file changed.** The brief says “stop the running container if running”. Always stopping would make the second run report changes, so I guarded it. Tell me if you want it unguarded and accept changed=2.
- **Privileges.** The playbook uses sudo (`become`). On a developer machine add `-K` or `-e use_become=false`.
- **Networking.** The container uses host networking so MQTT_HOST=localhost reaches the truck's Mosquitto; change mqtt_host if your broker is elsewhere.
- **Prerequisites on the target:** Docker, the Python docker package and the community.docker collection.